CSCE-5683
Digital Image Processing

PyTorch

Spring 2026

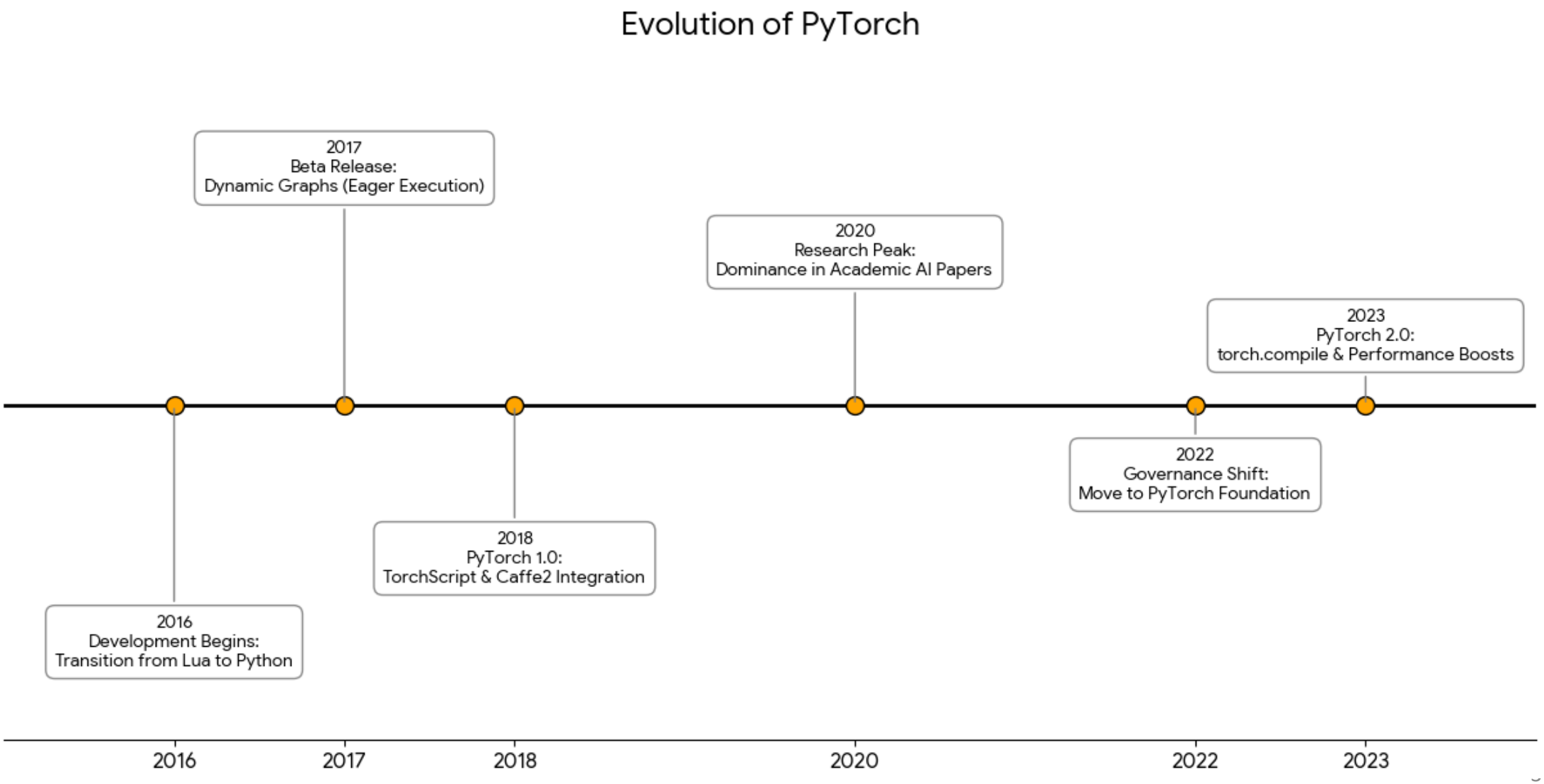
Prof. Khoa Luu

Prepared by Thanh-Dat Truong

PyTorch

- What is PyTorch?

PyTorch History

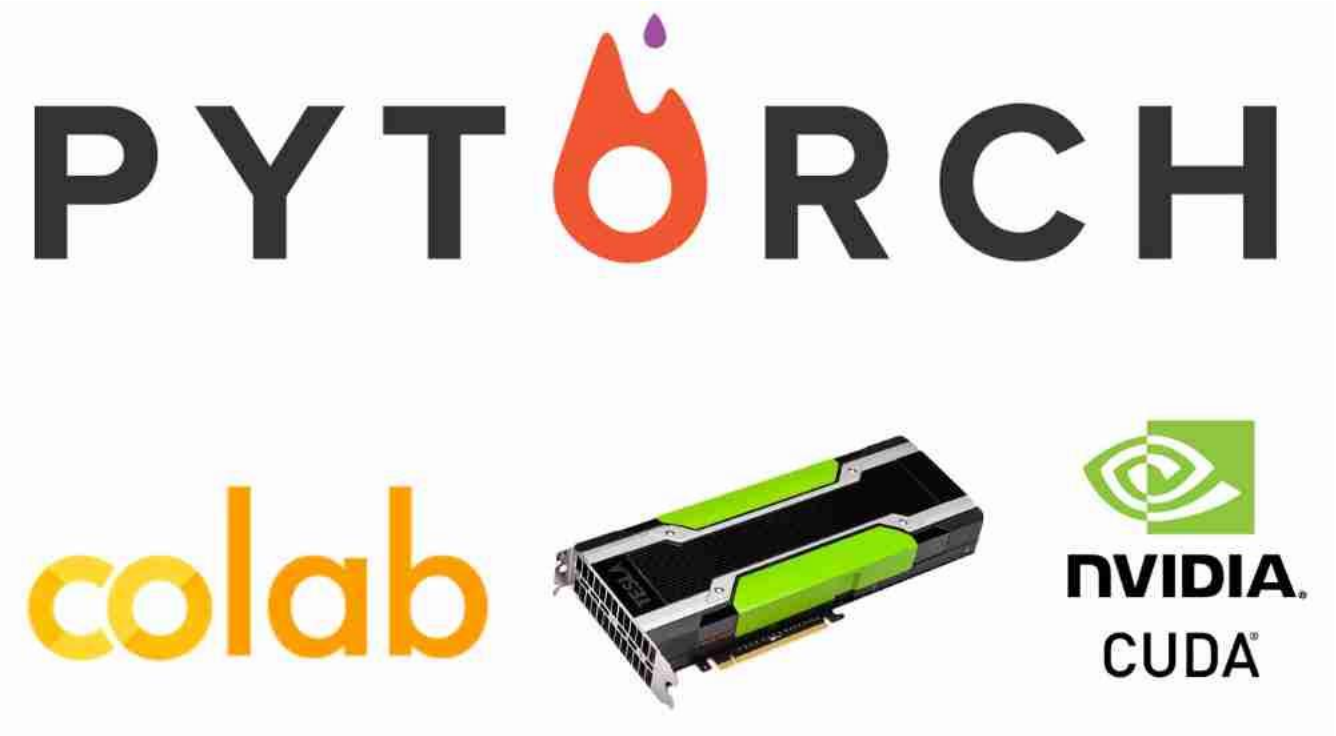


PyTorch

- What is PyTorch?
  - A deep learning framework
  - Developed by Facebook

PyTorch

- What is PyTorch?
  - A deep learning framework
  - Developed by Facebook
  - Support CUDA/CuDNN



PyTorch

- What is PyTorch?
  - A deep learning framework
  - Developed by Facebook
  - Support CUDA/CuDNN
  - Dynamic Graph

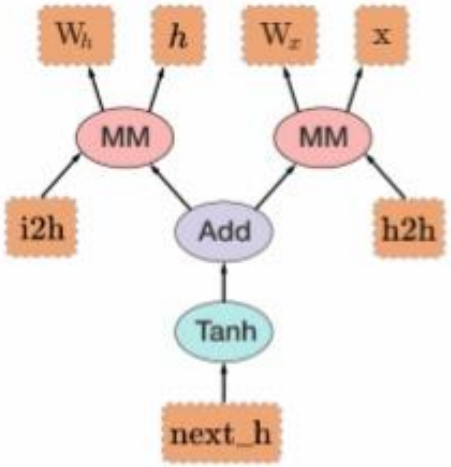
Dynamic Computational Graph

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

i2h = torch.mm(W_x, x.t())
h2h = torch.mm(W_h, prev_h.t())
next_h = i2h + h2h
next_h = next_h.tanh()
```

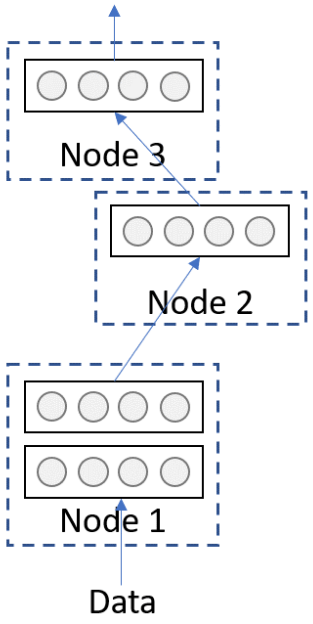


PyTorch

- What is PyTorch?
  - A deep learning framework
  - Developed by Facebook
  - Support CUDA/CuDNN
  - Dynamic Graph
  - C++ Support

PyTorch

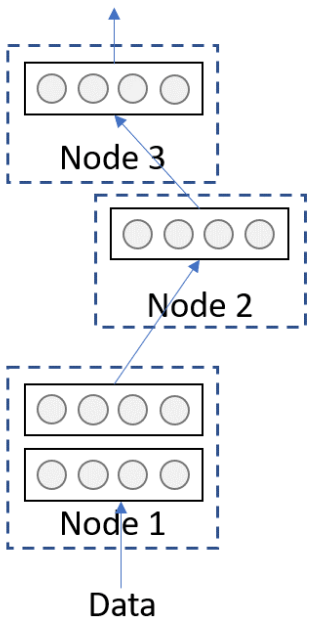
- What is PyTorch?
  - A deep learning framework
  - Developed by Facebook
  - Support CUDA/CuDNN
  - Dynamic Graph
  - C++ Support
  - Distributed Training
    - Model Parallelism



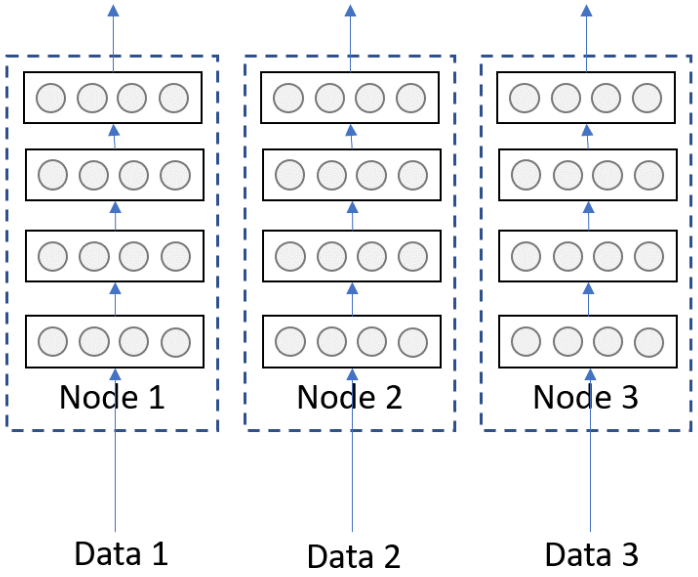
Model Parallelism

PyTorch

- What is PyTorch?
  - A deep learning framework
  - Developed by Facebook
  - Support CUDA/CuDNN
  - Dynamic Graph
  - C++ Support
  - Distributed Training
    - Model Parallelism
    - Data Parallelism



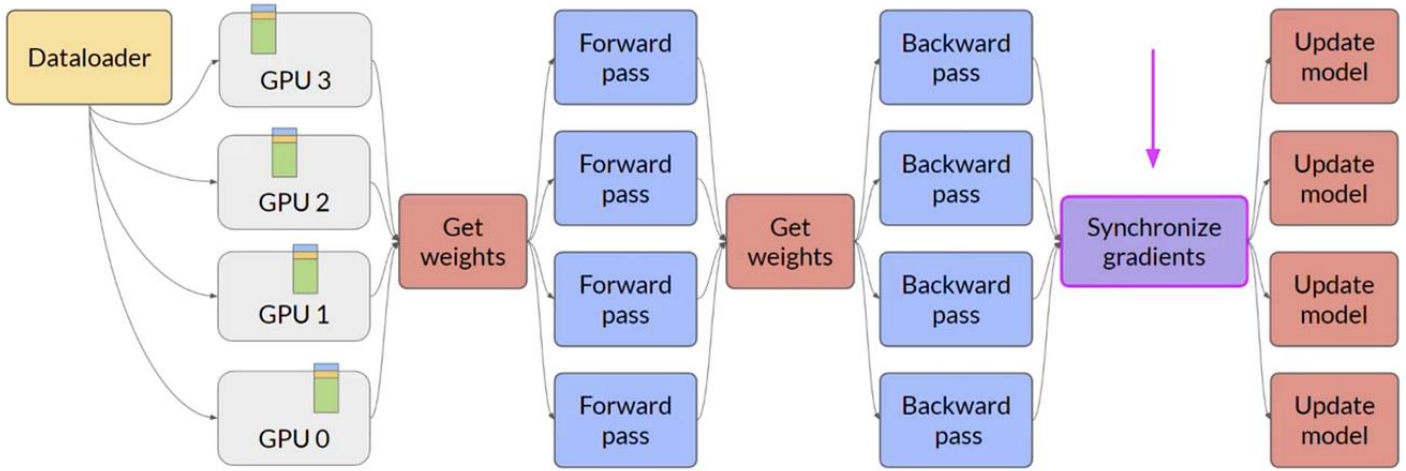
Model Parallelism



Data Parallelism

PyTorch

- What is PyTorch?
  - A deep learning framework
  - Developed by Facebook
  - Support CUDA/CuDNN
  - Dynamic Graph
  - C++ Support
  - Distributed Training
    - Model Parallelism
    - Data Parallelism
    - Fully Sharded Data Parallel

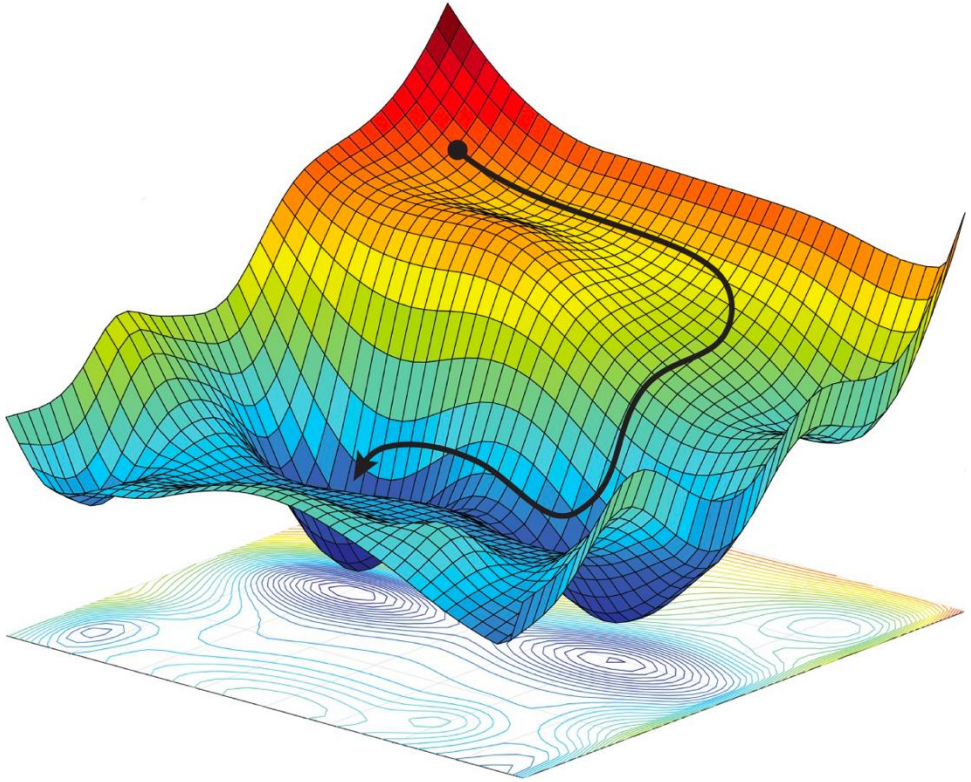


PyTorch

- Torch
  - Neural network/deep learning module

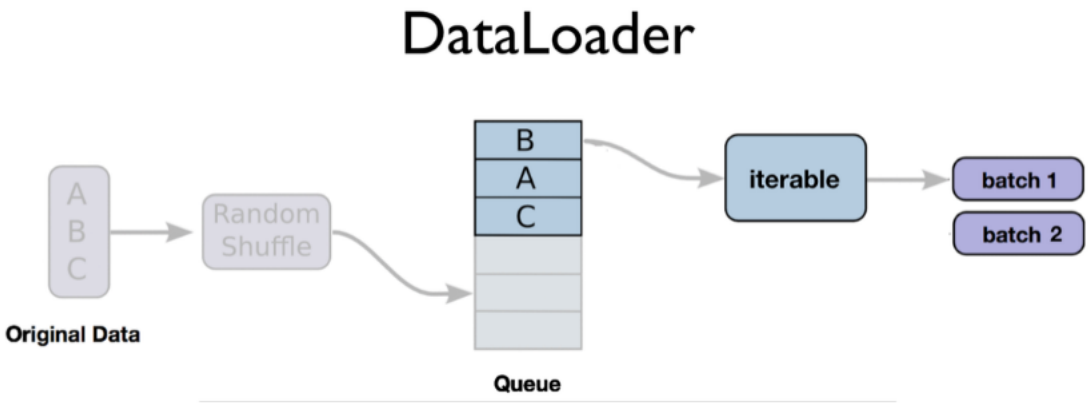
PyTorch

- Torch
  - Neural network/deep learning module
  - Optimization



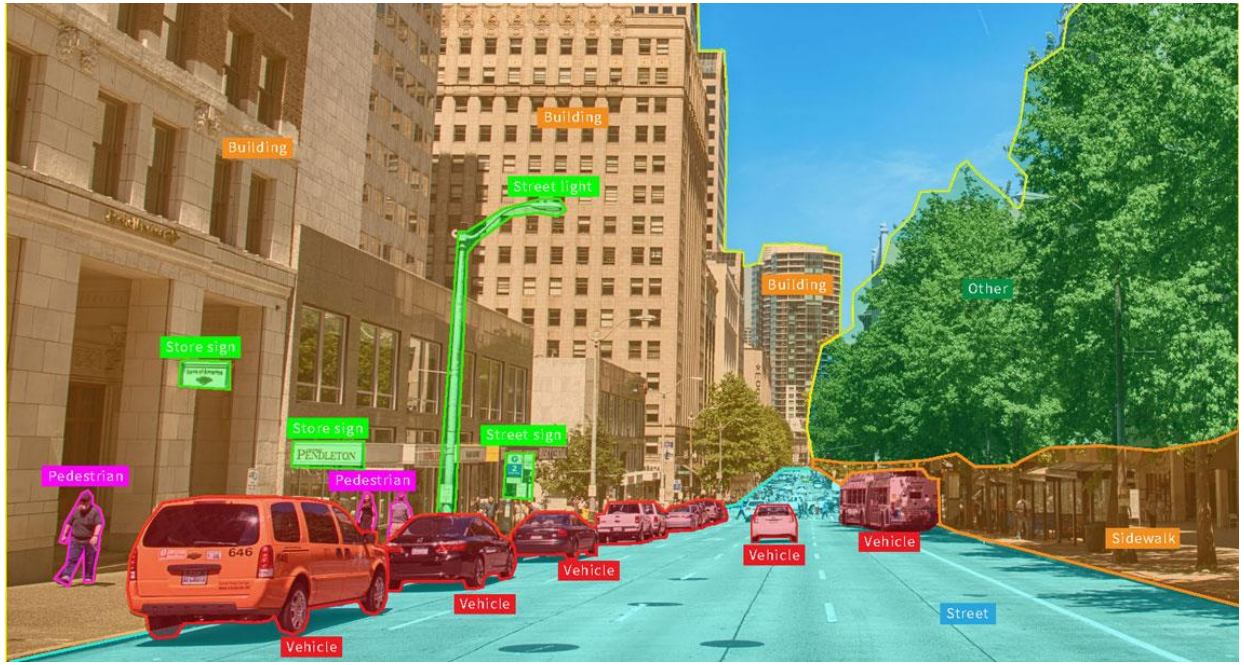
PyTorch

- Torch
  - Neural network/deep learning module
  - Optimization
  - Data Management



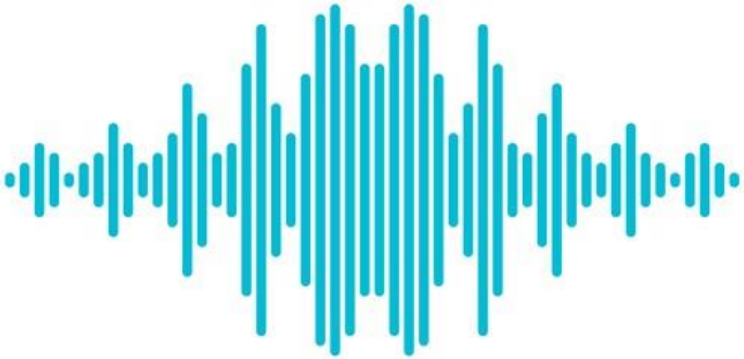
PyTorch

- Torch
  - Neural network/deep learning module
  - Optimization
  - Data Management
- Torchvision
  - Computer Vision



PyTorch

- Torch
  - Neural network/deep learning module
  - Optimization
  - Data Management
- Torchvision
  - Computer Vision
- Torchaudio
  - Audio Processing



PyTorch

- Torch
  - Neural network/deep learning module
  - Optimization
  - Data Management
- Torchvision
  - Computer Vision
- Torchaudio
  - Audio Processing
- PyTorch3D
  - 3D Data and Computer Grahics

PyTorch3D

Hugging Face

- Hugging Face Eco
  - Transformers
  - Accelerators
  - Diffusers
  - Perf
  - Datasets
  - And many more

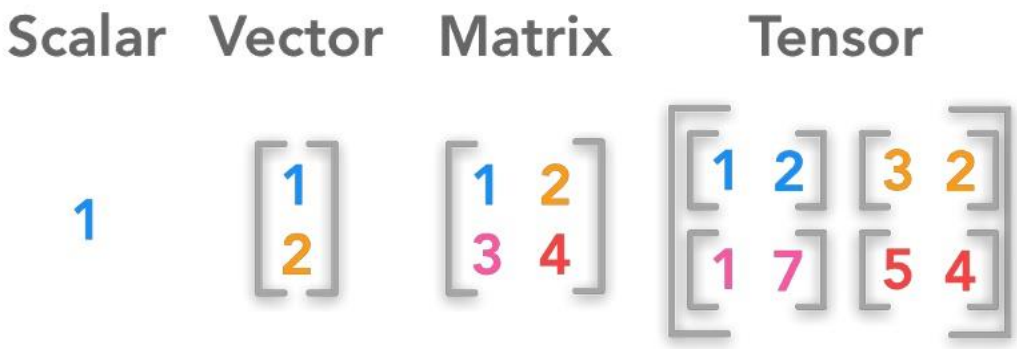
Hugging Face

PyTorch

- Install
  - By pip: **pip install torch torchvision**
- Verify torch version
  - Command line**: python -c “import torch; print(torch.__version__)”**

Tensor

- Tensor
  - An N-d array/matrix



Tensor

- Tensor
  - An N-d array/matrix
  - NumPy array style

```
# Numpy
a = np.ones(shape = (5, 5))
a[1, 2] = 4
b = np.random.randn(5, 5)
c = a + b
```

```
# PyTorch
a = torch.ones(5, 5)
a[1, 2 ] = 4
b = torch.randn(5, 5)
c = a + b
```

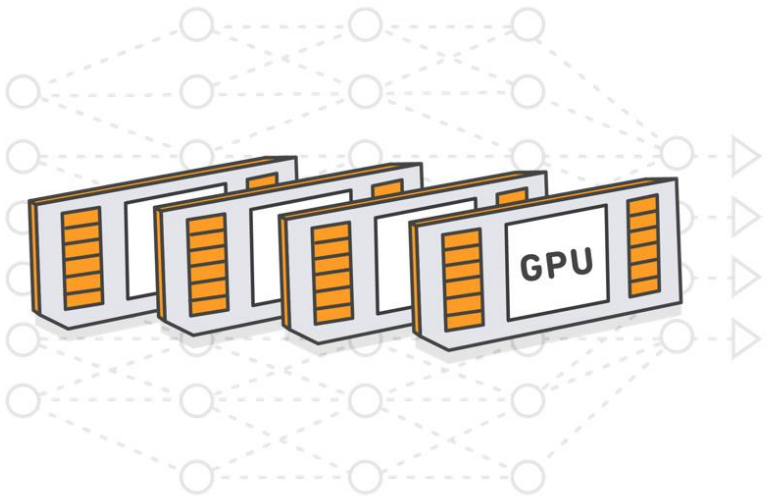
Tensor

- Tensor
  - An N-d array/matrix
  - NumPy array style
  - Easy converting between NumPy and Torch

```
# Initialize a numpy array
a = np.ones((5, 5))
# Convert a numpy array to a tensor
b = torch.from_numpy(a)
# Convert a tensor to a numpy array
c = b.numpy()
```

Tensor

- Tensor
  - An N-d array/matrix
  - NumPy array style
  - Support GPU to accelerate computing



GPU Tensor

- GPU Tensor
  - Check available

```
if torch.cuda.is_available():
```

GPU Tensor

- GPU Tensor
  - Check available
  - Get Device

```
if torch.cuda.is_available():
    device = torch.device("cuda")
```

GPU Tensor

- GPU Tensor
  - Check available
  - Get Device
  - Create tensors on GPU

```
if torch.cuda.is_available():
    device = torch.device("cuda")
    y = torch.ones_like(x, device=device)
    x = x.to(device) # or x.to("cuda")
```

GPU Tensor

- GPU Tensor
  - Check available
  - Get Device
  - Create tensors on GPU
  - z = x + y → x and y must be the same device

```
if torch.cuda.is_available():
    device = torch.device("cuda")
    y = torch.ones_like(x, device=device)
    x = x.to(device) # or x.to("cuda")
    z = x + y
```

GPU Tensor

- GPU Tensor
  - Check available
  - Get Device
  - Create tensors on GPU
  - z = x + y → x and y must be the same device
  - Copy z from GPU to CPU

```
if torch.cuda.is_available():
    device = torch.device("cuda")
    y = torch.ones_like(x, device=device)
    x = x.to(device) # or x.to("cuda")
    z = x + y
    z = z.to("cpu", torch.double)
```

Dataset and Data Loader

- Dataset
  - Define via OOP (class)

```
import torch
import torchvision
from PIL import Imgage # Use to read image

class CustomDataset(torch.utils.data.Dataset):
```

Dataset and Data Loader

- Dataset
  - Define via OOP (class)
  - Define __init__

```
import torch
import torchvision
from PIL import Imgage # Use to read image

class CustomDataset(torch.utils.data.Dataset):
    def __init__(self, arg1, arg2):
        super(CustomDataset, self).__init__()
        # YOUR INITILIZATION
        # ...
```

Dataset and Data Loader

- Dataset
  - Define via OOP (class)
  - Define __init__
  - Define __len__

```
import torch
import torchvision
from PIL import Imgage # Use to read image

class CustomDataset(torch.utils.data.Dataset):
    def __init__(self, arg1, arg2):
        super(CustomDataset, self).__init__()
        # YOUR INITILIZATION
        # ...
    def __len__(self):
        # Return the number of samples
```

Dataset and Data Loader

- Dataset
  - Define via OOP (class)
  - Define __init__
  - Define __len__
  - Define __getitem__

```
import torch
import torchvision
from PIL import Imgage # Use to read image

class CustomDataset(torch.utils.data.Dataset):
    def __init__(self, arg1, arg2):
        super(CustomDataset, self).__init__()
        # YOUR INITILIZATION
        # ...
    def __len__(self):
        # Return the number of samples
    def __getitem__(self, index):
        # Return the index-th sample of the dataset
```

Dataset and Data Loader

- Dataset
  - Torchvision support common datasets
    - MNIST, CIFAR-10, CIFAR-100, ImageNet

```
train_set = torchvision.datasets.CIFAR10(root="data",
              train = True,
              download = True,
              transform=torchvision.transforms.ToTensor())
```

Dataset and Data Loader

- Dataset
  - Torchvision support common datasets
    - MNIST, CIFAR-10, CIFAR-100, ImageNet

```
train_set = torchvision.datasets.CIFAR10(root="data",
              train = True,
              download = True,
              transform=torchvision.transforms.ToTensor())
```

Data directory

Dataset and Data Loader

- Dataset
  - Torchvision support common datasets
    - MNIST, CIFAR-10, CIFAR-100, ImageNet

```
train_set = torchvision.datasets.CIFAR10(root="data",
            train = True,
            download = True,
            transform=torchvision.transforms.ToTensor())
```

Data directory

Training set (true) or Testing set (false)

Dataset and Data Loader

- Dataset
  - Torchvision support common datasets
    - MNIST, CIFAR-10, CIFAR-100, ImageNet

```
train_set = torchvision.datasets.CIFAR10(root="data",
            train = True,
            download = True,
            transform=torchvision.transforms.ToTensor())
```

Data directory

Training set (true) or Testing set (false)

Automatically download dataset

Dataset and Data Loader

- Dataset
  - Torchvision support common datasets
    - MNIST, CIFAR-10, CIFAR-100, ImageNet

```
train_set = torchvision.datasets.CIFAR10(root="data",
            train = True,
            download = True,
            transform=torchvision.transforms.ToTensor())
```

Data directory

Training set (true) or Testing set (false)

Automatically download dataset

Convert PIL Image to Tensor

Dataset and Data Loader

- Dataset
  - Load samples (and labels) from disks
- Data Loader
  - Pack samples (and labels) into a mini batch

Dataset and Data Loader

- Dataset
  - Load samples (and labels) from disk
- Data Loader
  - Pack samples (and labels) into a mini batch

```
train_loader = torch.utils.data.DataLoader(train_set,
                    batch_size = 8,
                    shuffle = True,
                    num_workers = 4)
```

Our dataset

Dataset and Data Loader

- Dataset
  - Load samples (and labels) from disk
- Data Loader
  - Pack samples (and labels) into a mini batch

```
train_loader = torch.utils.data.DataLoader(train_set,
                    batch_size = 8,
                    shuffle = True,
                    num_workers = 4)
```

Our dataset

Batch size

Dataset and Data Loader

- Dataset
  - Load samples (and labels) from disk
- Data Loader
  - Pack samples (and labels) into a mini batch

```
train_loader = torch.utils.data.DataLoader(train_set,
                    batch_size = 8,
                    shuffle = True,
                    num_workers = 4)
```

Our dataset

Batch size

Shuffle dataset or not

Dataset and Data Loader

- Dataset
  - Load samples (and labels) from disk
- Data Loader
  - Pack samples (and labels) into a mini batch

```
train_loader = torch.utils.data.DataLoader(train_set,
                    batch_size = 8,
                    shuffle = True,
                    num_workers = 4)
```

Our dataset

Batch size

Shuffle dataset or not

Number of workers (or number of threads)

PyTorch Tensor vs NumPy Array

- What does make different between PyTorch and Numpy?

PyTorch Tensor vs NumPy Array

- What does make different between PyTorch and Numpy?
  - GPU Support

PyTorch Tensor vs NumPy Array

- What does make different between PyTorch and Numpy?
  - GPU Support
  - Autograd: Automatic Differentiation

Autograd

- Why do we need autograd?

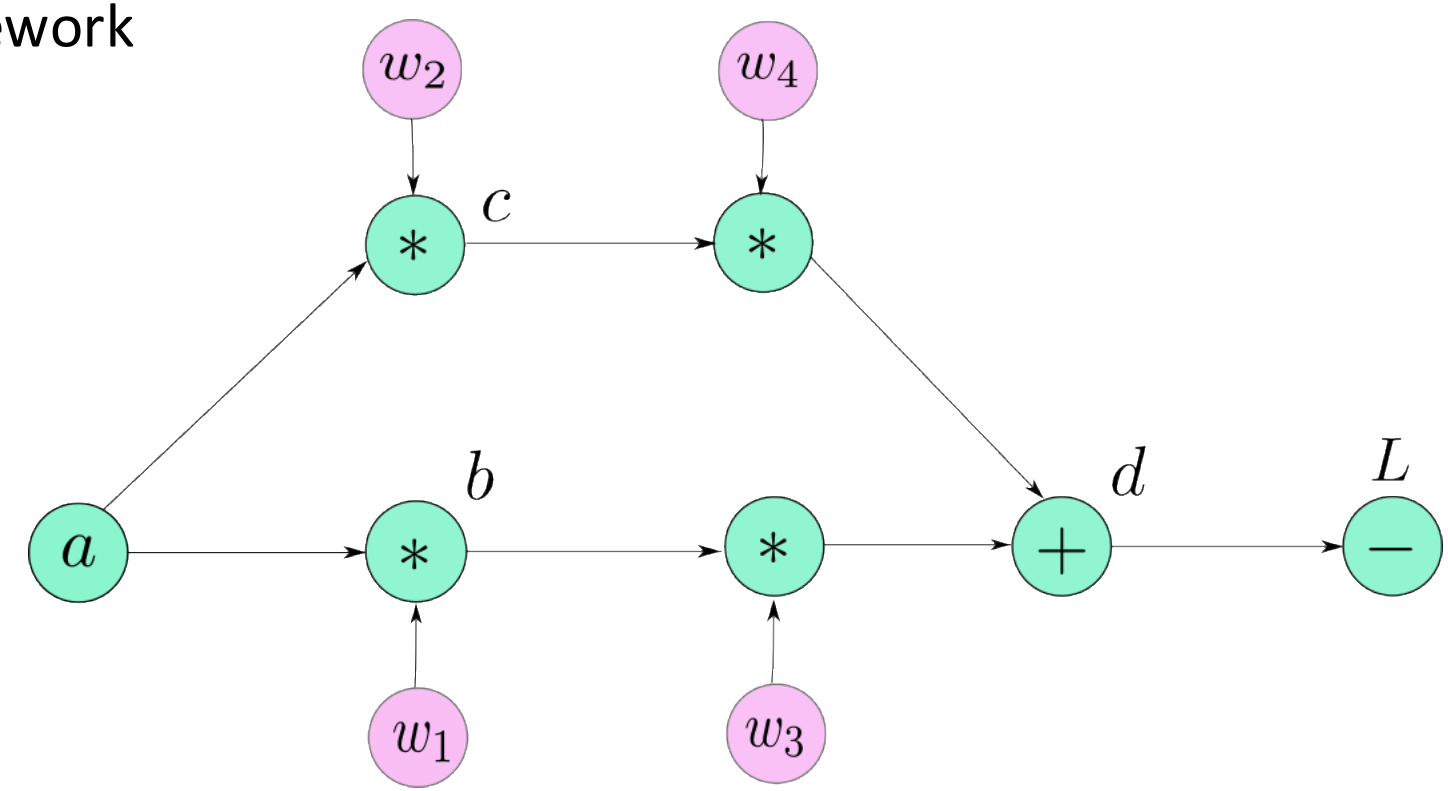
Autograd

- Why do we need autograd?
  - Learning neural network requires gradients

Autograd

- Why do we need autograd?
  - Learning neural network requires gradients
  - Define-by-run framework

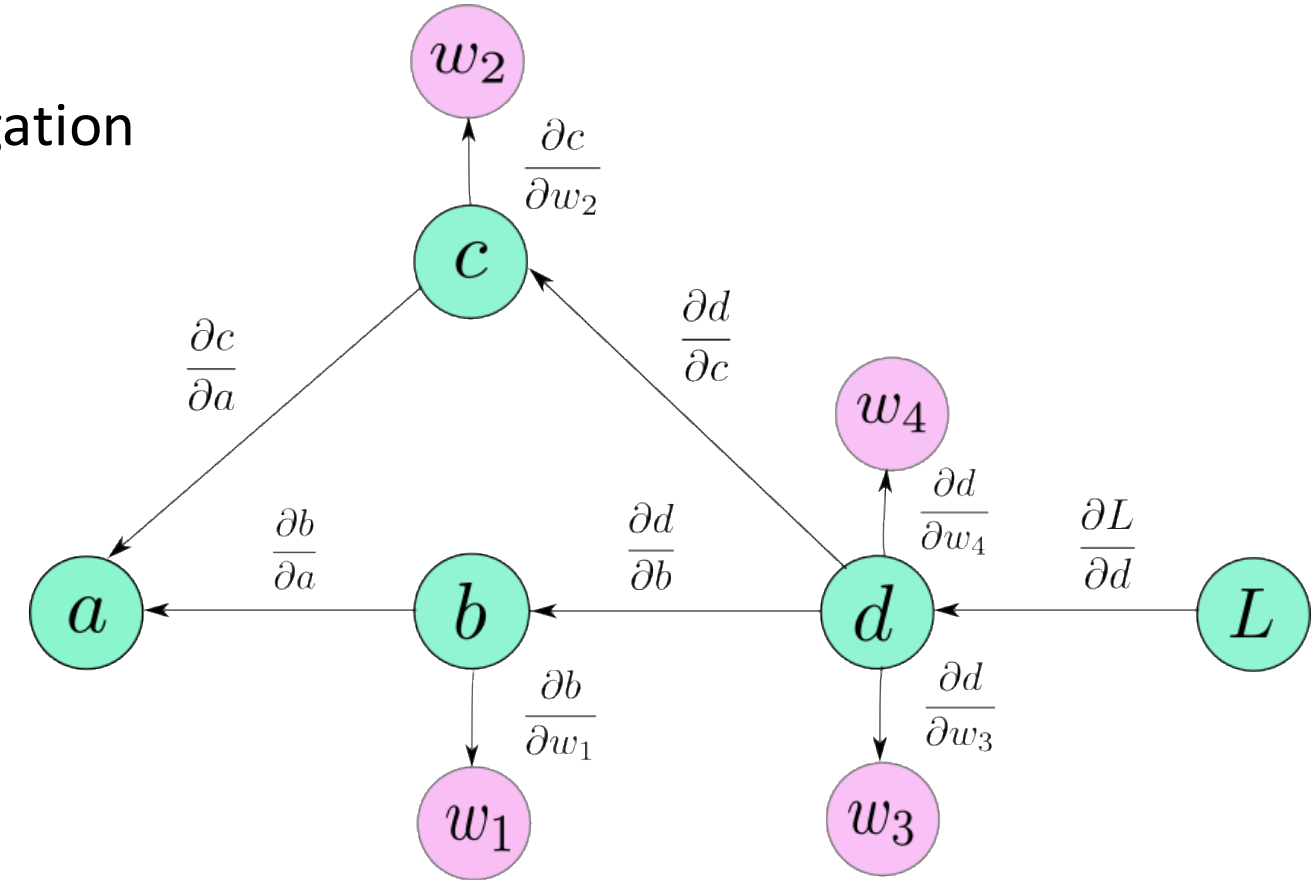
$$b = a * w_1$$
$$c = a * w_2$$
$$d = b * w_3 + c * w_4$$
$$L = -d$$



Autograd

- Why do we need autograd?
  - Learning neural network requires gradients
  - Define-by-run framework
  - Automatically Backpropagation

$$b = a * w_1$$
$$c = a * w_2$$
$$d = b * w_3 + c * w_4$$
$$L = -d$$



Autograd

- Examples
  - Set its attribute .requires_grad as **True**

```
x = torch.ones(2, 2, requires_grad=True)
```

Autograd

- Examples
  - Set its attribute .requires_grad as **True**
  - Call .backward() and have all the gradients computed automatically.

```
x = torch.ones(2, 2, requires_grad=True)
y = x + 2
z = y * y * 3
out = z.mean()
out.backward()
```

Autograd

- Examples
  - Set its attribute .requires_grad as **True**
  - Call .backward() and have all the gradients computed automatically
  - Gradient of tensors will be accumulated into .grad attribute

```
x = torch.ones(2, 2, requires_grad=True)
y = x + 2
z = y * y * 3
out = z.mean()
out.backward()
print(x.grad)
```

[Output]: [[4.5, 4.5], [4.5, 4.5]]

Autograd

- Examples
  - Set its attribute .requires_grad as **True**
  - Call .backward() and have all the gradients computed automatically
  - Gradient of tensors will be accumulated into .grad attribute
  - Call .detach() to detach it from the computation history

```
x = torch.ones(2, 2, requires_grad=True)
y = x + 2
z = y * y * 3
out = z.mean()
out.backward()
print(x.grad)
x = x.detach()
```

[Output]: [[4.5, 4.5], [4.5, 4.5]]

Neural Network

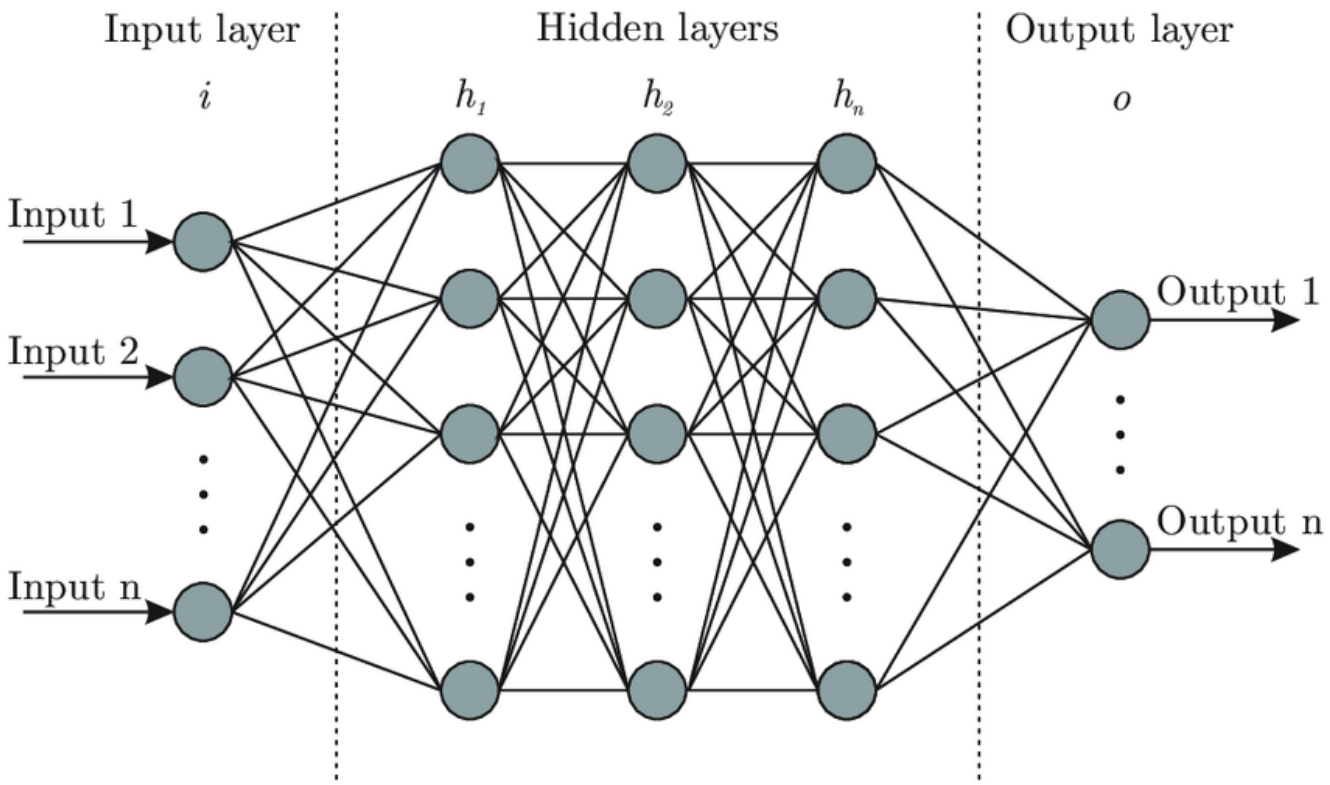
- What is a neural network?

Neural Network

- What is a neural network?
  - Inspired by the biological neural networks

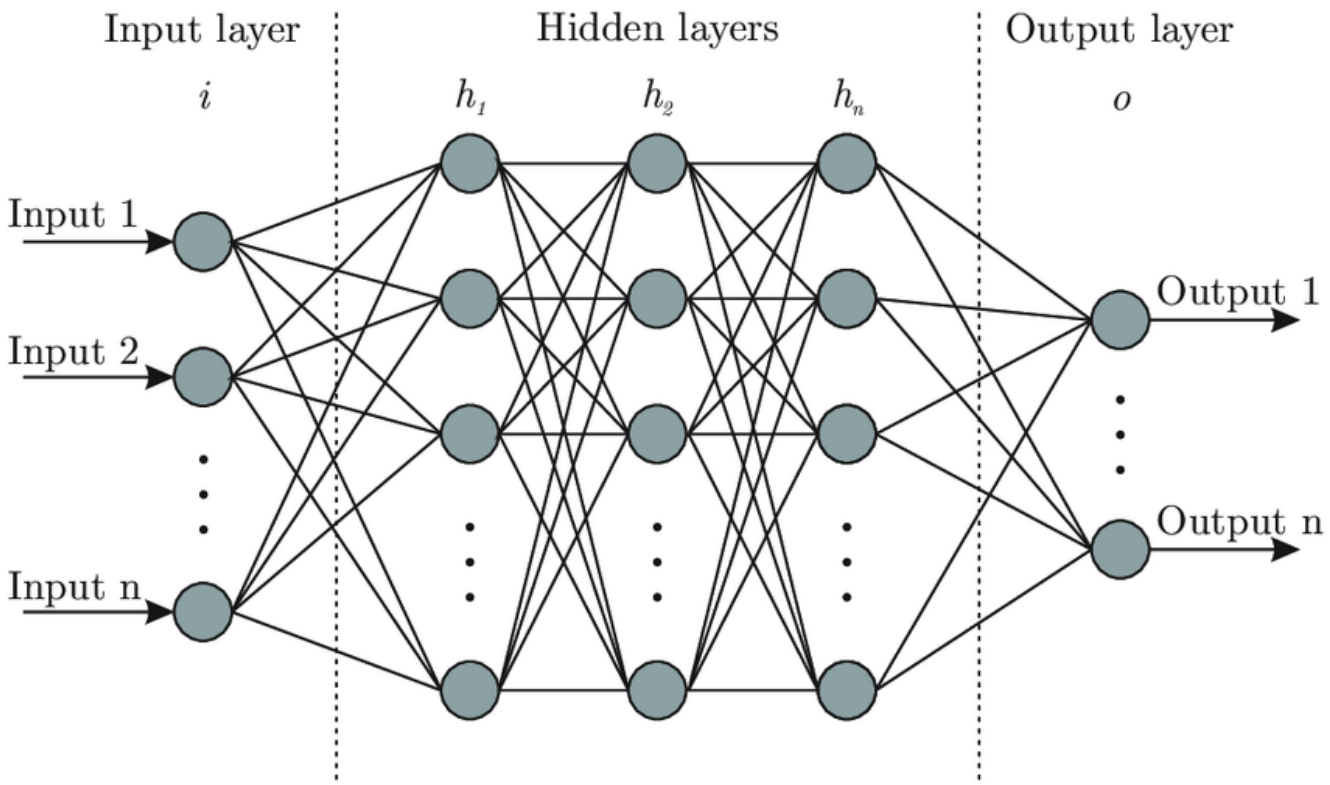
Neural Network

- What is a neural network?
  - Inspired by the biological neural networks
  - Consist of a stack of layers



Neural Network

- What is a neural network?
  - Inspired by the biological neural networks
  - Consist of a stack of layers



Neural Network

- How to define a neural network?
  - Constructed by using torch.nn

```
class SampleNet(torch.nn.Module):
```

Neural Network

- How to define a neural network?
  - Constructed by using torch.nn
  - Define layers

```
class SampleNet(torch.nn.Module):
  def __init__(self, D_in, H, D_out):
    super(SampleNet, self).__init__()
    self.linear1 = torch.nn.Linear(D_in, H)
    self.linear2 = torch.nn.Linear(H, D_out)
```

Neural Network

- How to define a neural network?
  - Constructed by using torch.nn
  - Define layers
  - Define forward function

```
class SampleNet(torch.nn.Module):
  def __init__(self, D_in, H, D_out):
    super(SampleNet, self).__init__()
    self.linear1 = torch.nn.Linear(D_in, H)
    self.linear2 = torch.nn.Linear(H, D_out)

  def forward(self, x):
    h_relu = self.linear1(x).clamp(min=0)
    y_pred = self.linear2(h_relu)
    return y_pred
```

Training Neural Network

- Typical training procedure:
  - Define the neural network

```
net = SampleNet()
```

Training Neural Network

- Typical training procedure:
  - Define the neural network
  - Iterate over a dataset of inputs

```
net = SampleNet()
for inputs, targets in dataloader:
```

Training Neural Network

- Typical training procedure:
  - Define the neural network
  - Iterate over a dataset of inputs
  - Forward input through the network

```
net = SampleNet()
for inputs, targets in dataloader:
  outputs = net(inputs)
```

Training Neural Network

- Typical training procedure:
  - Define the neural network
  - Iterate over a dataset of inputs
  - Forward input through the network
  - Compute loss

```
net = SampleNet()
for inputs, targets in dataloader:
  outputs = net(inputs)
  loss = loss_fn(outputs, targets)
```

Training Neural Network

- Typical training procedure:
  - Define the neural network
  - Iterate over a dataset of inputs
  - Forward input through the network
  - Compute loss
  - Propagate gradients

```
net = SampleNet()
for inputs, targets in dataloader:
  outputs = net(inputs)
  loss = loss_fn(outputs, targets)
  net.zero_grad()
  loss.backward()
```

Training Neural Network

- Typical training procedure:
  - Define the neural network
  - Iterate over a dataset of inputs
  - Forward input through the network
  - Compute loss
  - Propagate gradients
  - Update the weights of the network
    - ***weight = weight - learning_rate * gradient***

```
net = SampleNet()
for inputs, targets in dataloader:
  outputs = net(inputs)
  loss = loss_fn(outputs, targets)
  net.zero_grad()
  loss.backward()
  lr = 0.01
  for f in net.parameters():
    f.data.sub_(f.grad.data * lr)
```

Training Neural Network

- What does the procedure look like?

```
net = SampleNet()
for inputs, targets in dataloader:
  outputs = net(inputs)
  loss = loss_fn(outputs, targets)
  net.zero_grad()
  loss.backward()
  lr = 0.01
  for f in net.parameters():
    f.data.sub_(f.grad.data * lr)
```

Training Neural Network

- What does the procedure look like?
  - Stochastic Gradient Descent

```
net = SampleNet()
for inputs, targets in dataloader:
  outputs = net(inputs)
  loss = loss_fn(outputs, targets)
  net.zero_grad()
  loss.backward()
  lr = 0.01
  for f in net.parameters():
    f.data.sub_(f.grad.data * lr)
```

Training Neural Network

- What does the procedure look like?
  - Stochastic Gradient Descent (SGD)
- PyTorch Optimization
  - Various optimization rules such as SGD

```
net = SampleNet()
for inputs, targets in dataloader:
  outputs = net(inputs)
  loss = loss_fn(outputs, targets)
  net.zero_grad()
  loss.backward()
  lr = 0.01
  for f in net.parameters():
    f.data.sub_(f.grad.data * lr)
```

Training Neural Network

- What does the procedure look like?
  - Stochastic Gradient Descent
- PyTorch Optimization
  - Various optimization rules such as SGD
  - Include in torch.optim

```
import torch.optim as optim
net = SampleNet()
params = net.parameters()
optimizer = optim.SGD(params, lr=0.01)
for inputs, targets in dataloader:
  outputs = net(inputs)
  loss = loss_fn(outputs, targets)
  optimizer.zero_grad()
  loss.backward()
  optimizer.step()
```

Let's start with Training a Classifier

- Step 1: Define Dataset and Data Loader

```
train_set = torchvision.datasets.CIFAR10(root="data",
                    train = True,
                    download = True,
                    transform=torchvision.transforms.ToTensor())
test_set = torchvision.datasets.CIFAR10(root="data",
                    train = False,
                    download = True,
                    transform=torchvision.transforms.ToTensor())
train_loader = torch.utils.data.DataLoader(train_set,
                    batch_size = 32,
                    shuffle = True,
                    num_workers = 4)
test_loader = torch.utils.data.DataLoader(test_set,
                    batch_size = 32,
                    shuffle = True,
                    num_workers = 4)
```

Let's start with Training a Classifier

- Step 2: Define our Neural Network

```
import torch.nn as nn
import torch.nn.functional as F
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 3)
        self.conv2 = nn.Conv2d(6, 16, 3)
        self.fc1 = nn.Linear(16 * 6 * 6, 120) # 6*6 from image dimension
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

Let's start with Training a Classifier

- Step 2: Define our Neural Network

```
import torch.nn as nn
import torch.nn.functional as F
class Net(nn.Module):
    def __init__(self):
        ...

    def forward(self, x):
        x = F.max_pool2d(F.relu(self.conv1(x)), (2, 2))
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)
        x = x.reshape(x.size(0), -1) # Flatten Features
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Let's start with Training a Classifier

- Step 2: Define our Neural Network

```
import torch.nn as nn
import torch.nn.functional as F
class Net(nn.Module):
    def __init__(self):
        ...

    def forward(self, x):
        ...
net = Net()
print(net)
```

Let's start with Training a Classifier

- Step 2: Define our Neural Network

```
import torch.nn as nn
import torch.nn.functional as F
class Net(nn.Module):
    def __init__(self):
        ...

    def forward(self, x):
        ...
net = Net()
print(net)
```

```
Net(
  (conv1): Conv2d(3, 6, kernel_size=(3, 3), stride=(1, 1))
  (conv2): Conv2d(6, 16, kernel_size=(3, 3), stride=(1, 1))
  (fc1): Linear(in_features=576, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
```

Let's start with Training a Classifier

- Step 3: Define optimizer and Loss Function

```
learning_rate = 0.001
optimizer = torch.optim.SGD(net.parameters(), lr=learning_rate)
n_epochs = 2
cuda = torch.cuda.is_available() # Check availability of CUDA
if cuda:
    net.cuda()
loss_fn = torch.nn.CrossEntropyLoss()
```

Let's start with Training a Classifier

- Step 4: Train our model

```
net.train() # Set training mode
for epoch in range(n_epochs):
  for iter, (images, labels) in enumerate(train_loader):
    if cuda:
      images = images.cuda()
      labels = labels.cuda()
    outputs = net(images)
    loss = loss_fn(outputs, labels)
    accuracy = (torch.argmax(output, dim=1) == labels).float().mean() * 100.0
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if iter % 20 == 0:
    print("Epoch [%d/%d]. Iter [%d/%d]. Loss: %0.4f. Accuracy: %0.2f" %
       (epoch + 1, n_epochs, iter + 1, len(train_loader), loss, accuracy))
```

Let's start with Training a Classifier

- Step 5: Test our model

```
net.eval() # Set testing mode
final_accuracy = 0.0
for iter, (images, labels) in enumerate(test_loader):
    if cuda:
        images = images.cuda()
        labels = labels.cuda()
    outputs = net(images)
    accuracy = (torch.argmax(outputs, dim=1) == labels).float().mean() * 100.0
    final_accuracy += accuracy.item()
    if iter % 20 == 0:
    print("Iter [%d/%d]. Accuracy: %0.2f" %
                            (iter + 1, len(train_loader), accuracy))

final_accuracy /= len(test_loader)
print("Accuracy: %0.2f" % final_accuracy)
```

Google Colab

- What is Google Colab?

Google Colab

- What is Google Colab?
  - A cloud service

Google Colab

- What is Google Colab?
  - A cloud service
  - A Python Notebook

Google Colab

- What is Google Colab?
  - A cloud service
  - A Python Notebook
  - GPU Support

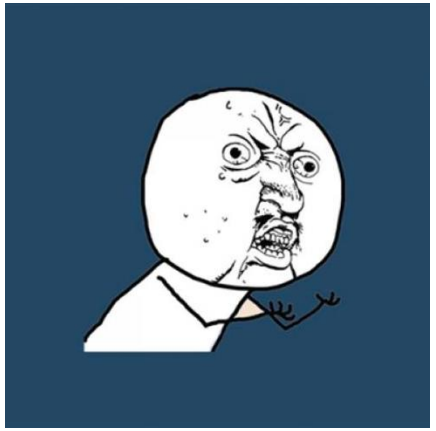
Google Colab

- What is Google Colab?
  - A cloud service
  - A Python Notebook
  - GPU Support
  - Of course, it's ***FREE***

Google Colab

- What is Google Collab?
  - A cloud service
  - A Python Notebook
  - GPU Support
  - Of course, it's ***FREE***

    (it's free within continuous 12 hours)

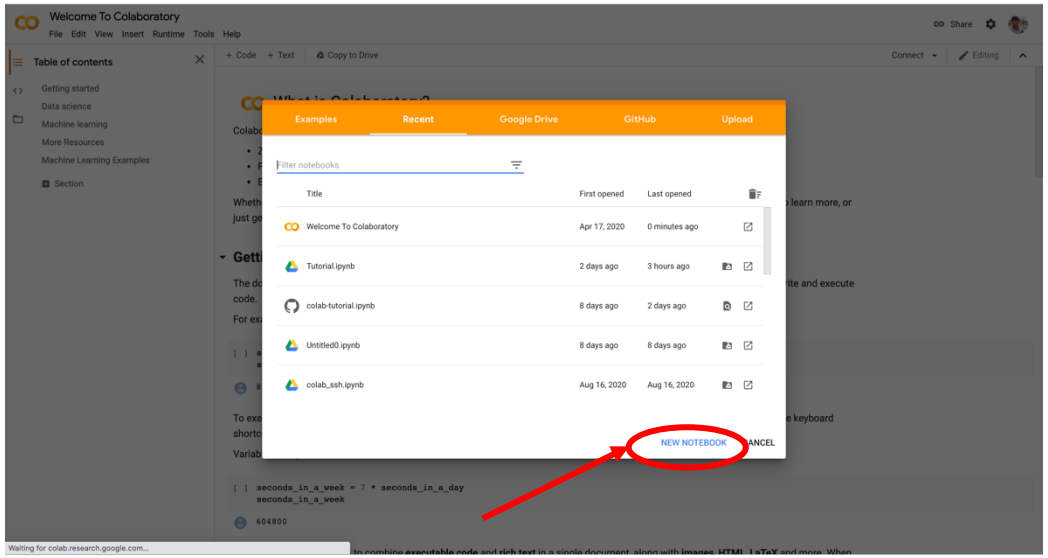


How to Use Google Colab

- To use Google Colab
  - Access to https://colab.research.google.com/

How to Use Google Colab

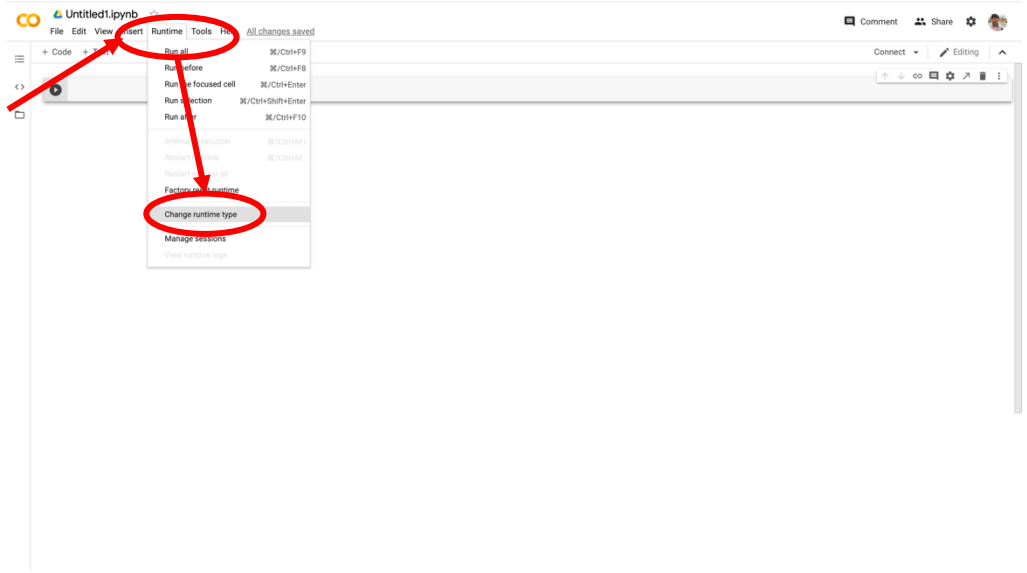
- To use Google Colab
  - Access to https://colab.research.google.com/
  - Choose New Notebook



How to Use Google Colab

- To use Google Colab
  - Access to https://colab.research.google.com/
  - Choose **New Notebook**

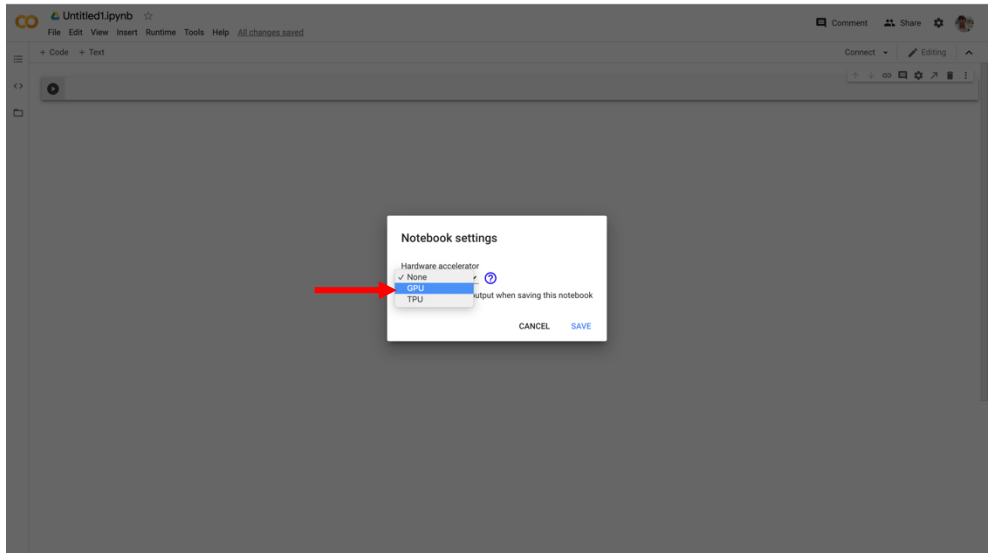
- To use GPU Environment
  - Select **Runtime → Change Runtime Type**



How to Use Google Colab

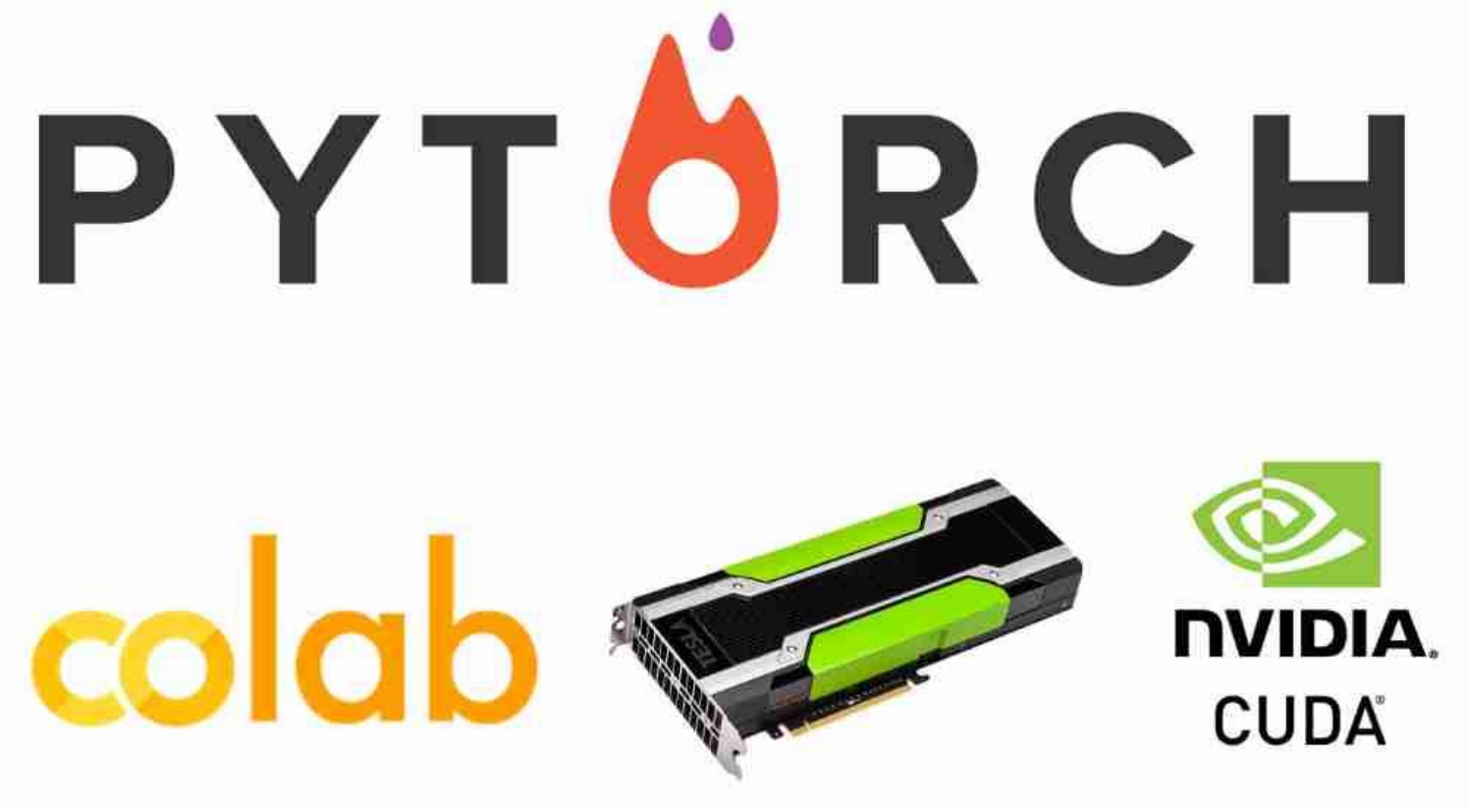
- To use Google Colab
  - Access to https://colab.research.google.com/
  - Choose New Notebook

- To use GPU Environment
  - Select Runtime → Change Runtime Type
  - Select **GPU**



PyTorch

- Install PyTorch on Google Colab
  - Already Support



Practice PyTorch in Google Colab

- Let’s practice PyTorch now
  - https://colab.research.google.com/drive/1BBQDwT_m8r6_Kh62oLxcig_PaBX-VBxz?usp=sharing